

Part 2 — Running the crawler on billions of URLs

Summary

Part 1 is a service that takes one URL and returns the page's metadata, type and topics. Part 2 runs that code over a customer's monthly list of billions of URLs, stores the results, and serves them to millions of requests.

The design is a pipeline. The list is loaded once a month into a **frontier**, a table of every waiting URL with a priority. A **scheduler** releases URLs only as fast as each website allows. A fleet of **workers** runs the Part 1 code on each one. Raw HTML goes to object storage; metadata and extracted text go to a column-oriented database that serves both customer reports and single-URL lookups.

Two facts shape the design:

- **The shape of the list decides the limit.** Billions of URLs spread over a million websites are limited by CPU for parsing, so the design is about fleet size. Billions concentrated on three websites, as in the brief's example, are limited by how fast those sites tolerate being fetched, and no fleet size changes that; the design is then about choosing which fraction of the list to spend the budget on.
- **The parse is the cost.** Fetching waits on the network; reading a page costs half a second of CPU. Every cost lever here parses fewer pages or parses them cheaper.

Rounded estimates at 10 billion URLs a month over many sites: about 260 servers, about \$84,000 a month at first and about \$120,000 by the second year as storage accumulates, roughly \$8.4 per million URLs. For the three-site example the same machinery runs on two servers on a floor of about \$2,000 a month and fetches about 78 million pages a month per site until the sites agree to more. All figures are back-of-envelope, rounded to two figures and good to about $\pm 50\%$ until the proof of concept measures page size, parse cost and how often pages change. Calculations are shown where they are used.

1. What the brief asks for

The brief asks for "a service that will take billions of URL and identify the metadata on the page, allow millions of requests on the content, and optimize for cost, performance and availability". The input is "a list of billions of URLs send in via a text file and/or in MySQL for a given year month", with the example "billions of URLs for amazon.com, walmart.com, bestbuy.com etc for July". The required outputs are a storage design for metadata and content, a unified schema, SLOs and SLAs, and the monitoring metrics and tools.

This document uses **host** for what the brief calls a domain, **list** for what the customer sends, **batch** for one month's run of that list, and **fetch** for retrieving one URL.

2. Two kinds of input

	Many hosts	Few hosts (the brief's example)
Shape of the list	billions of URLs over roughly a million websites	billions of URLs over three websites
What limits throughput	CPU to parse pages	the request rate each website tolerates
What the design must do	size and feed a fleet	spend a fixed per-site budget on the most valuable URLs, and report what was not reached

The arithmetic for the few-host case is short. A site that has not agreed to anything tolerates on the order of 10 requests a second from one crawler; that is an assumption, at the upper end of common practice. Ten a second is 26 million fetches a month. Against 3.3 billion URLs per site, that is under 1% of the list each month, and the whole list would take over ten years. At a negotiated 100 requests a second it is still over a year. No number of servers changes this. In that regime the system's job is to pick the right 1% and to tell the customer the coverage it achieved; §4.6 designs the picking.

The many-host case is the one that needs a fleet, and it is sized in §3.

3. How big is it

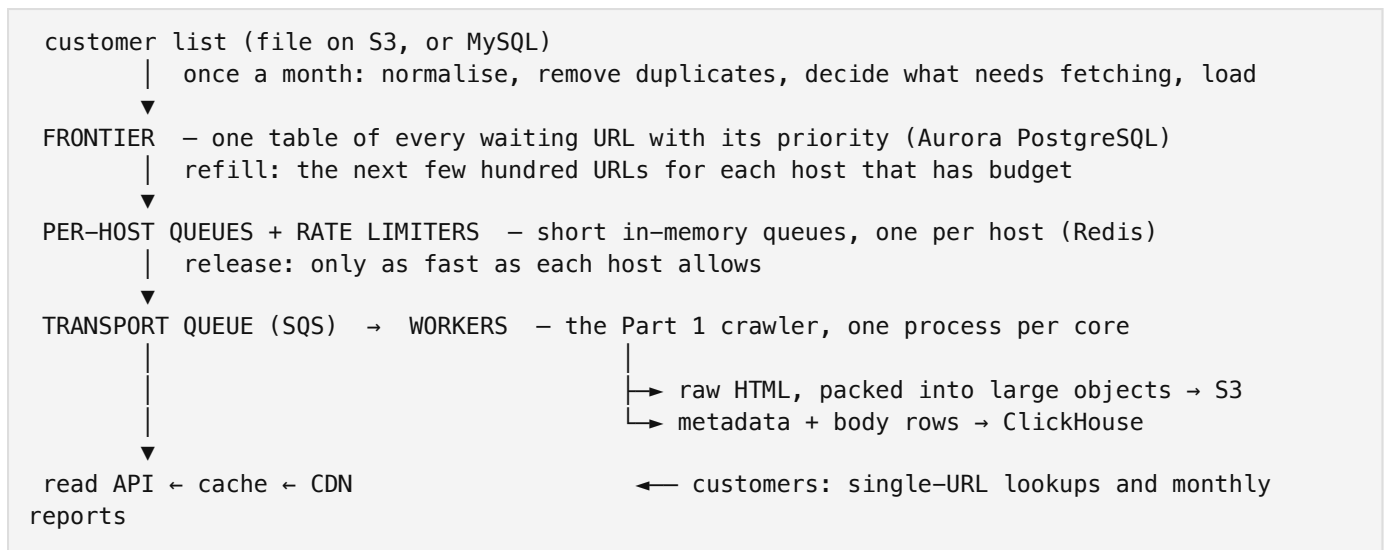
Planning volume: 10 billion URLs a month, an assumed upper case; at 2 billion, divide everything linear by five.

Quantity	Calculation	Result
Fetches per second, sustained	$10 \text{ billion} \div (30 \text{ days} \times 86,400 \text{ s})$	~3,900 /s
CPU per page	0.5 s measured on a laptop (two large fixture pages, five runs each) $\times$ 1.5 for a slower server core	0.75 s, assumed
Cores needed	$3,900 \text{ /s} \times 0.75 \text{ s}$	~2,900
Servers deployed	$2,900 \div 16 \text{ cores} \div 70\% \text{ utilisation (headroom)}$	~260 of 16 cores
Fetches in flight	$3,900 \text{ /s} \times 2 \text{ s per fetch, assumed}$	~7,700; ~30 per server
Raw HTML stored per month	$10 \text{ billion} \times 200 \text{ KB (assumed)} \div 5 \text{ (compression, measured)}$	~400 TB
Metadata and body stored per month	$10 \text{ billion} \times 10 \text{ KB (measured)} \div 4 \text{ (assumed)}$	~25 TB
Frontier table	$10 \text{ billion rows} \times \sim 120 \text{ bytes}$	~1.2 TB

The least certain number is page size. The two fixture pages are 1.6 MB and 5.6 MB, far above the 200 KB planning value, and if the real average is 1 MB then storage and bandwidth are five times larger. The second least certain is parse cost on the real server. Both are the first things the proof of concept measures.

Memory is not a constraint: a worker process uses about 400 MB after a parse, so sixteen of them fit in a 32 GB server with room to spare.

4. The system



4.1 How a URL travels

1. The list arrives as a file or a database table. Each URL is normalised and hashed; duplicates within the list are dropped by sorting; each URL is compared with what has already been crawled, given a priority, and loaded into the frontier.
2. The scheduler pulls the next few hundred URLs, by priority, for each host that has budget into that host's short in-memory queue.
3. It releases URLs from those queues onto the transport queue only as fast as each host's rate limiter permits, so the transport queue holds seconds of work, never a backlog for a host that is out of budget.
4. A worker takes a URL, checks the host's limiter once more, fetches the page, runs the Part 1 pipeline, appends the raw HTML to its current output object and writes the metadata row.
5. A blocked, empty or error page is recorded as a row too. Nothing is silently dropped.

4.2 The frontier

The frontier is a table in Aurora PostgreSQL, partitioned by host, with one row per waiting URL: host, URL hash, the customer's weight, when it is next due, and when it was last released. About 1.2 TB for 10 billion rows.

A table rather than a queue because the fetch order changes over time: a page that changed last time should come back sooner, one that has not changed in months can wait. A queue is fixed once written; a table makes the order a query, "for this host, the URLs that are due, most valuable first". The due date is written once per fetch, so nothing is rewritten in bulk.

A database cannot answer "next URL for this host" 3,900 times a second, so the scheduler pulls a few hundred URLs per host at a time into a short Redis queue and releases from there. Redis also holds each host's rate limiter. Both are rebuildable from the table, so losing Redis loses no URLs: anything released over an hour ago without a result is released again, and a repeated fetch is harmless. Each host's queue holds at most ten minutes of its rate, so a million hosts fit in a few gigabytes.

This is the URL-frontier pattern from Alex Xu's *System Design Interview* (vol. 1, ch. 9): prioritiser, priority-ordered front queues, one back queue per host, and a selector, with most of the frontier on disk and only the heads in memory. The one difference is a table instead of files for the disk tier, so priorities can change after loading.

4.3 Politeness

Each host has a rate cap: its `robots.txt` crawl delay, a rate the customer negotiated, or a default of 10 requests a second. The limiter enforces it at release and the worker checks it again before fetching, so a backlog can never

burst onto one host. A host that answers 429 or starts failing has its rate halved and recovers slowly. `robots.txt` is fetched once per host, cached for a day and obeyed per URL. Blocks are recorded, never evaded: the crawler identifies itself and does not pretend to be a browser.

4.4 Workers

A worker server runs one process per core, each with a few fetches in flight: fetching waits on the network, parsing holds a core for half a second, and one process per core is the safe unit. Each server runs a local DNS cache, because 3,900 lookups a second across a million hosts would saturate the cloud resolver. Servers are Graviton Spot instances at about a third of the on-demand price; an interruption costs nothing because every URL is an idempotent unit of work.

4.5 Storage

Three kinds of data, three stores, chosen by access pattern.

Data	Access pattern	Store
Raw HTML	written once, read rarely (re-extraction)	S3, packed 1,000 pages per object, tiered to cheaper classes with age
Metadata and extracted body	scanned by host and month; looked up by URL	ClickHouse, two tables (below)
Frontier, hosts, rate policies	transactional	Aurora PostgreSQL

Raw HTML is kept so the extractor can be re-run when it improves, which is far cheaper than fetching again. Each worker packs the pages it fetches into one object per minute; a page is addressed by object key, offset and length. Packing matters: one object per page would be 3,900 writes a second and about \$50,000 a month in request charges alone; packed, it is a few writes a second and about \$50.

Unified schema. One table holds every page type; product-specific fields such as price or SKU go in a key-value column rather than separate tables. The important columns:

Column	Meaning
<code>batch_month</code> , <code>fetches_at</code>	the month the customer asked for, and when the fetch actually happened
<code>host</code> , <code>url_hash</code> , <code>url</code> , <code>final_url</code>	identity, before and after redirects
<code>status</code> , <code>reason</code> , <code>robots_state</code> , <code>http_status</code>	what happened: success, blocked or error, and why
<code>title</code> , <code>description</code> , <code>canonical_url</code> , <code>author</code> , <code>published_date</code> , <code>language</code> , <code>headings</code> , <code>Open Graph</code> and <code>Twitter tags</code>	the metadata
<code>body</code> , <code>word_count</code>	the extracted content
<code>page_type</code> , <code>topics</code> (ranked)	classification and topics
<code>s3_key</code> , <code>s3_offset</code> , <code>s3_length</code>	where the raw HTML is
<code>content_hash</code> , <code>extractor_version</code>	for change detection and re-extraction
<code>extra</code> (map)	type-specific fields

Two ClickHouse tables carry it. `page_fetch` keeps every fetch, including blocked and failed ones, partitioned by day; it is the history and the audit trail. `page_current` keeps the latest row per URL, fed automatically from the first, partitioned by URL hash so repeat fetches collapse onto one row. Both are sorted by host then URL hash, which

serves the customer's "all of amazon.com for July" scan and the "this one URL" lookup from the same store. Full definitions are in §10.

4.6 Not fetching what you don't need

Three questions, each answered exactly. A Bloom filter is not used: at 1% false positives it would silently drop a hundred million URLs a month as "already seen".

- **Seen in this list?** Sort by hash at load time and drop neighbours.
- **Crawled before, and due again?** Join the list against `page_current` at load time.
- **Changed since last time?** After the fetch, hash the raw bytes; if unchanged, skip the parse and the write. This is the largest saving because the parse is the cost. The planning assumption is 60% unchanged, but Amazon served a different variant of the same page on each of three fetches, so for hosts like that the saving may be near zero; the proof of concept measures it per host.

Choosing what to fetch when the budget is a fraction of the list (the few-host case): each URL carries the customer's weight and a due date. The interval starts from the page type (articles weekly, products monthly), halves when the *extracted* content changed and doubles when it did not, between one day and ninety. Release order is weight, then due date. New URLs and recrawls share the budget half and half so a large new list does not starve freshness. Fixed schedules such as "news hourly" are not used; at any budget they starve the rest of the host.

4.7 Serving millions of requests

Assume 10 million reads a day, about 120 a second and twice that at peak: single-URL lookups and analytical reports. Both go to ClickHouse; a lookup on the sorted table reads one small block. A Redis cache takes 90% of repeats; a CDN caches at the edge and terminates TLS. A key-value copy of every record was rejected: 10 billion records of 10 KB into DynamoDB is 100 billion write units, about \$120,000 a month, to serve roughly a dozen cache misses a second. If measured lookup latency proves unacceptable, a key-value tier of one-kilobyte summaries returns at a tenth of that.

API surface: GET `/pages/{url}` for one URL's latest row; GET `/hosts/{host}/pages?month=` for a host and month, paginated by URL hash; GET `/hosts/{host}/summary?month=` for counts by status, reason and page type. Reads carry an API key with a per-key rate limit; the cache key is the normalised URL, invalidated when `page_current` changes.

4.8 Pages that are not the page

Measured on the test URLs: Amazon served the product page, CNN the article, and REI answered 403 for the page and for `robots.txt` while serving the page to a browser's User-Agent. So coverage is measured per host, and two things stay apart everywhere: **blocked** (the site refused) is a coverage number; **error** (something failed) is a reliability number.

Some blocks are disguised. Myntra served a "Site Maintenance" page with a 200 to the crawler and the product page to a browser. Part 1 catches these generically: a 200 with almost no visible text and no metadata is `no_content`. At fleet scale two more signals separate a disguised block from an outage: the same content hash on many URLs of one host is a template page, and a browser-User-Agent probe that succeeds where the crawler failed is a block.

Pages that need JavaScript to render are excluded in the first version and counted per host; rendering, at ten to twenty times the CPU of a static parse, is budgeted only if that share is material. SLAs are written for fetchable pages and report the excluded counts.

5. When things fail

Everything is idempotent, so the answer to almost every failure is "do it again".

Failure	What happens
Worker dies mid-fetch	the queue message reappears; the URL is fetched again
Spot instance reclaimed	two minutes' notice: stop taking work, flush the output object, exit
Redis lost	limiters rebuilt from policy, queues refilled from the frontier; anything released over an hour ago without a result is released again
Frontier table unavailable	no refills; queues drain within minutes and the fleet idles visibly; nothing lost
A page crashes the parser	after three attempts it is dead-lettered and recorded as <code>parse_failed</code>
ClickHouse falls behind	workers buffer for a few minutes, then the scheduler slows releases
A host starts blocking	its rate halves; an alert and a coverage note to the customer
Bad deploy	roll back; deploys freeze while the error budget recovers

Back-pressure always flows toward the frontier, the one place that can slow the system without losing anything.

Availability. Stateful components run across three zones in one region: Aurora with a synchronous standby (failover under a minute, no data loss), Redis with a replica per shard (rebuildable anyway), ClickHouse with two replicas of every part, and S3 by construction. Workers and the API are stateless across zones. The 99.9% read objective in §6 rests on that layout plus the cache and CDN, which keep serving during a database failover. A fetch in flight during a failure is fetched again. Loss of a whole region is not covered: a second region is a later step (§9) and the residual risk is stated in the SLA.

6. Objectives and commitments

Internal objectives (SLOs), measured continuously:

Objective	Many hosts	Few hosts
Released URL fetched	99% within 1 hour	same
Batch completion	99% of fetchable URLs within the month plus a day	at least 95% of each host's agreed daily budget used
Parse latency	95th percentile under 1.5 s	same
Permanent errors (not blocks)	under 0.5%	same
Row visible to queries	within 10 minutes of fetch	same
Read API	99.9% available; cached reads under 200 ms	same

External commitments (SLAs), deliberately looser: API 99.5% monthly; 95% of *fetchable* URLs per batch, with blocked, disallowed and JavaScript-only counts reported per host; for the few-host case, coverage at the agreed rate with a per-host estimate of completion rather than a percentage of the list. "Fetchable" is the operative word: promising a share of *all* URLs would be promising to defeat bot detection.

7. Monitoring

Per-host figures are computed in ClickHouse from the fetch table, not emitted as metric labels, because a million hosts would overwhelm a metrics database. Fleet metrics go to Prometheus and Grafana; queue and autoscaling signals to CloudWatch; a sample of fetches is traced end to end.

Metric	Threshold	Action
Batch progress and per-host completion estimate	estimate past the deadline	if fleet-limited, scale; if host-limited, report to the customer
Age of oldest released message	over 1 hour	add 20% capacity, up to 1.5× the mean
Block rate per host	2× its baseline	coverage ticket; customer report
Error rate per host	over 5% for 15 minutes	check host health; pause if 5xx
no_content rate per host; many URLs sharing one hash	over 1%; over 100 URLs	probe with a browser User-Agent; if it succeeds, count the host as blocking
Parse latency and parse failures	p95 over 1.5 s; failures over 1%	page size growth or a parser regression
Unchanged rate on recrawl	drops 10 points week on week	normalisation or hashing regression; cost rises directly
ClickHouse parts and insert lag	over 300 parts; lag over 5 minutes	larger insert batches; slow the scheduler
Cost per million URLs	over 20% above model	check the unchanged rate, page size, instance mix

Every worker writes one structured log event per fetch with the URL, reason, robots state and HTTP status; that event and the row it accompanies are the source for all of the above.

8. What it costs

Prices are public list prices for us-east-1 as recalled at writing; every figure below is an estimate to within about $\pm 50\%$ until the proof of concept replaces the assumptions with measurements.

Assumptions: 10 billion URLs a month; 200 KB per page, compressing 5:1; 0.75 s of CPU per page on a server core; Graviton Spot at \$0.02 per core-hour; 10 KB per metadata record; storage tiered with age.

Many hosts, with the calculation behind each line:

Line	Calculation	Per month
Worker fleet	260 servers × 16 cores × 720 h × \$0.02 per Spot core-hour	~\$59,000
Databases, caches, ingest, API, monitoring	ClickHouse 6 nodes with two replicas, Aurora with its I/O, Redis, ingest job, API, monitoring, at list prices	~\$9,500
Egress, queue, object writes	260 public IPs × \$0.005/h (\$930); 3 queue requests per URL, batched (\$1,200); 1,000 pages per object write (\$50)	~\$2,200
Raw HTML storage	400 TB/month × \$0.023/GB in the first month, tiered to cheaper classes with age	\$9,200 in month 1; \$28,000 by month 24
Metadata storage	25 TB/month × 2 replicas × \$0.08/GB hot for three months, then cold	\$4,000 in month 1; \$24,000 by month 24

Line	Calculation	Per month
Total		~\$84,000 in month 1; ~\$120,000 by month 24; about \$8.4 per million URLs at the start

Few hosts: the same components on two worker servers: a floor of about \$2,000 a month for databases, caches and monitoring, and roughly \$27 per million URLs at 78 million a month. Cost is not the constraint in this regime; the sites' tolerance is.

The levers, which matter more than the totals:

Decision	Effect	Why
Skip the parse when the raw page is unchanged	up to –60% of the fleet, about \$36,000 a month at the planning rate	the parse is the cost; the saving is bounded by how often pages actually change, which Amazon's per-fetch variants show can be near zero
Pack raw pages into large objects	–99.9% of object-write charges: \$50 instead of ~\$50,000 a month	1,000 pages per write instead of one
Public IP egress instead of a NAT gateway	–95% of egress cost: \$930 instead of ~\$18,000 a month	NAT charges \$0.045 per GB; ~400 TB a month of compressed pages
Tier storage with age	–80% on data older than three months	raw HTML is read rarely after the re-extraction window
Bill instances at realistic utilisation	+40% versus a naive core count	servers are paid for whether busy or not; the model runs them at 70%
No key-value replica for serving	avoids ~\$120,000 a month in write charges	the sorted column store already answers single-URL lookups

9. What to validate first

In the order that would change the design most if wrong:

1. The request rate the target hosts accept, probed at 1, 5 and 10 a second; it decides the regime and everything promised.
2. Page-size distribution; storage and bandwidth scale with it.
3. Parse cost on the real server type; it sets the fleet.
4. How often pages are unchanged on recrawl, per host; it sets the largest saving.
5. Block rate and JavaScript-only share per host; they set the coverage ceiling.
6. ClickHouse at 3,900 inserts a second, and lookup latency under that load.
7. Failover drills for Aurora, Redis and a ClickHouse replica under load.
8. Frontier refill latency at a million hosts on a 1.2 TB table, and re-release after a Redis failover.
9. A fair column-store versus row-store comparison on real hardware before quoting any speed multiple.

10. Decisions and schema

Each decision names what was chosen, the alternatives and the property that ruled each out, and the condition that would reverse it; what to measure first for each is in §9. None of these components has been operated by the author in production; the choices are architectural.

#	Decision	Chosen	Rejected, and why	Reverses if
1	Transport queue	SQS, batched	Kafka: ordering and replay unused, partitions would track hosts. RabbitMQ: a queue per host and paid empty polls	several consumers need the same event
2	Rate limiters and per-host queues	Redis, Lua token bucket	DynamoDB: a write per token, ten times the cost of the queue itself and 10 ms per take. Postgres: a hot-row write per fetch	over ~50M hosts (shard)
3	Frontier store	Aurora PostgreSQL table, partitioned by host, priority computed at query time from stored weight and due date	sorted files on S3 read in order: the order cannot change after loading. All in Redis: ~1 TB of RAM	refill queries cannot keep up (pre-materialise heads)
4	Worker compute	Graviton Spot, one process per core	Lambda: roughly the Spot fleet's cost again, and no connection reuse. Fargate Spot: no control of process placement. EKS: a cluster to run an autoscaling group	Spot interruptions break the 1-hour objective (add on-demand baseline)
5	Raw store	S3, tiered	EFS: about thirteen times the price per GB. HDFS: storage tied to compute	frequent re-extraction (keep more hot)
6	Raw object packing	per (worker, minute), 1,000 pages	per page: 3,900 writes/s and a thousand times the request charges. Per (host, hour): at a million hosts each buffer holds one page	pages are rewritten often (they are not)
7	Metadata store	ClickHouse, two tables, two replicas	Postgres: reads every column of every row. BigQuery: per-TB-scanned pricing across clouds. Elasticsearch: 2–3× storage, not a scanner. DynamoDB: no scans	insert rate produces unmanageable parts
8	Schema	one table for all types; extra map; history by day, current by URL hash	one table by month: 10 billion rows in one partition and recrawls never collapse. Partition by host: too many parts	history never queried (keep 90 days)
9	Dedup	exact sort and join at load	Bloom filter: 1% false positives drop 100M URLs/month and cannot forget. Redis set: roughly half a terabyte of memory	URLs arrive as a stream
10	Serving	CDN → cache → ClickHouse	DynamoDB replica: about twice the worker fleet's cost, in write charges alone	lookup p99 over 100 ms under load
11	Egress	public IPv4 on workers	NAT gateway: charged per byte, about twenty times the public-IP cost. NAT instances: single-AZ	a partner needs a short allow-list (NAT with fixed IPs)
12	Observability	Prometheus + Grafana, CloudWatch, traces; per-host figures in ClickHouse	Datadog: per-host pricing. CloudWatch custom metrics per host: per-metric pricing × 1M	few-host regime (CloudWatch alone)
13	JavaScript rendering	excluded and measured	render all: fleet × 10–20. Ignore: hidden coverage gap	measured share over 10%
14	Recrawl policy	adaptive interval per URL inside a per-host budget	fixed cadence per type: "hourly" starves the rest of the host. Uniform monthly: no freshness	customer supplies priorities

Schema, ClickHouse:

```

CREATE TABLE page_fetch (
  batch_month Date, fetched_at DateTime64(3),
  host LowCardinality(String), url_hash UInt64, url String, final_url String,
  status LowCardinality(String), reason LowCardinality(String), robots_state
LowCardinality(String),
  http_status Nullable(UInt16), content_type LowCardinality(String), content_hash UInt64,
  title String, description String, canonical_url String, author String,
  published_date Nullable(DateTime), language LowCardinality(String), h1_headings
Array(String),
  og_tags Map(String, String), twitter_tags Map(String, String),
  body String CODEC(ZSTD(3)), word_count UInt32,
  page_type LowCardinality(String), topics Nested(topic String, rank_score Float32),
  s3_key String, s3_offset UInt32, s3_length UInt32,
  extractor_version LowCardinality(String), extra Map(String, String)
) ENGINE = MergeTree PARTITION BY toYYYYMMDD(fetched_at) ORDER BY (host, url_hash,
fetched_at);

CREATE TABLE page_current AS page_fetch
ENGINE = ReplacingMergeTree(fetched_at) PARTITION BY cityHash64(url_hash) % 64 ORDER BY (host,
url_hash);

```

Frontier, Aurora PostgreSQL: frontier(host, url_hash, url, customer_weight, due_at, released_at, batch_month), partitioned by host hash, indexed on (host, released_at, customer_weight DESC, due_at). Rows older than the batch after the current one are purged.